

HiDiTingV100 OpenHarmony Native 组件开发

用户指南

文档版本 00B02

发布日期 2025-07-30

前言

概述

本文档详细的描述了 HiDiTingV100 上 Native 组件和 API 接口的使用方法，用于指导用户开发 Native 组件。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B02	2025-07-30	新增 “ 3 HTTP 组件 ” 章节。
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 Location 组件.....	1
1.1 概述.....	1
1.2 功能描述.....	1
1.3 Native API 参考.....	3
1.3.1 Locator 类.....	3
1.3.1.1 GetInstance.....	3
1.3.1.2 IsLocationEnabled.....	4
1.3.1.3 EnableAbility.....	5
1.3.1.4 StartLocating.....	6
1.3.1.5 StopLocating.....	7
1.3.1.6 RegisterGnssStatusCallback.....	7
1.3.1.7 UnregisterGnssStatusCallback.....	8
1.3.1.8 RegisterNmeaMessageCallback.....	9
1.3.1.9 UnregisterNmeaMessageCallback.....	10
1.3.1.10 QuerySupportCoordinateSystemType.....	11
1.3.1.11 IsLocationPrivacyConfirmed.....	12
1.3.1.12 SetLocationPrivacyConfirmStatus.....	13
1.3.1.13 SendCommand.....	14
1.3.2 ILocatorCallback 类.....	14
1.3.2.1 OnLocationReport.....	15
1.3.2.2 OnLocatingStatusChange.....	15
1.3.2.3 OnErrorReport.....	16
1.3.3 IGnssStatusCallback 类.....	17
1.3.3.1 OnStatusChange.....	17
1.3.4 INmeaMessageCallback 类.....	18
1.3.4.1 OnMessageChange.....	18
1.4 数据类型、数据结构.....	19

1.4.1 Location	19
1.4.2 RequestConfig	25
1.4.3 SatelliteStatus	28
1.4.4 LocationCommand	32
1.4.5 LocationSourceType	32
1.4.6 LocationRequestScenario	33
1.4.7 LocationRequestPriority	34
1.4.8 LocationSatelliteConstellation	35
1.4.9 LocationSatelliteAdditionalInfo	37
1.4.10 LocationErr	38
1.4.11 LocationCoordinateSystemType	38
1.4.12 LocationLocatingStatus	39
1.5 错误码	40
1.6 GNSS 套片对接说明	40
1.6.1 GNSS Emulator 说明	41
1.6.2 GNSS 套片使用说明	41
1.6.2.1 编译切换	41
1.6.2.2 PGNSS 注入说明	42
1.7 开发指导	42
1.7.1 定位准备工作	42
1.7.2 查询支持坐标系	43
1.7.3 实现 GnsStatus 及订阅回调函数	44
1.7.4 实现 NmeaMessage 及订阅回调函数	44
1.7.5 实现定位回调函数及启动定位	45
1.7.6 取消订阅 GnsStatus 回调函数	46
1.7.7 取消订阅 NmeaMessage 回调函数	46
1.7.8 停止定位及退出	47
2 Sensor 组件	48
2.1 概述	48
2.2 功能描述	48
2.3 Native API 参考	50
2.3.1 GetAllSensors	50
2.3.2 SubscribeSensor	51
2.3.3 UnsubscribeSensor	52
2.3.4 SetBatch	53

2.3.5 ActivateSensor.....	54
2.3.6 DeactivateSensor	55
2.3.7 SetMode	56
2.4 数据类型、数据结构.....	57
2.4.1 SensorTypeld.....	58
2.4.2 SensorInfo	61
2.4.3 SensorEvent	62
2.4.4 RecordSensorCallback.....	63
2.4.5 UserData.....	63
2.4.6 SensorUser.....	64
2.4.7 SensorMode	65
2.4.8 AccelData	66
2.4.9 GyroData	66
2.4.10 PressureData.....	67
2.4.11 SENSOR_NAME_MAX_LEN	68
2.4.12 SENSOR_USER_DATA_SIZE	68
2.4.13 VERSION_MAX_LEN.....	69
2.5 错误码	69
2.6 Sensor 驱动对接说明.....	69
2.7 开发指导.....	70
2.7.1 获取设备支持的 Sensor 列表	70
2.7.2 创建传感器订阅回调函数及传感器数据解析	70
2.7.3 创建传感器用户.....	73
2.7.4 使能传感器.....	73
2.7.5 订阅传感器数据.....	74
2.7.6 取消传感器数据订阅.....	74
2.7.7 去使能一个传感器	74
3 HTTP 组件	75
3.1 Native API 参考	75
3.1.1 HttpClientGetRequest.....	75
3.1.2 HttpClientPostRequest	76
3.1.3 HttpClientHeadRequest.....	77
3.1.4 HttpClientPutRequest	78
3.1.5 HttpClientDelete.....	79
3.1.6 HttpClientConn	80
3.1.7 HttpClientSend.....	81
3.1.8 HttpClientRecvResponse	82

3.1.9 HttpClientClose	82
3.1.10 HttpClientGetResponseCode	83
3.1.11 HttpClientSetCustomHeader	84
3.2 数据类型，数据结构	85
3.2.1 HTTP Request 类型	85
3.2.2 HTTP Client 结构体	85
3.2.3 HttpClientData 结构体	87
3.3 错误码	88

插图目录

图 1-1 定位子系统架构图 2

图 2-1 传感器子系统架构图 49

表格目录

表 1-1 定位子系统 API 错误码 40

表 2-1 传感器子系统 API 错误码..... 69

表 3-1 HTTP API 错误码 89

1 Location 组件

- 1.1 概述
- 1.2 功能描述
- 1.3 Native API 参考
- 1.4 数据类型、数据结构
- 1.5 错误码
- 1.6 GNSS 套片对接说明
- 1.7 开发指导

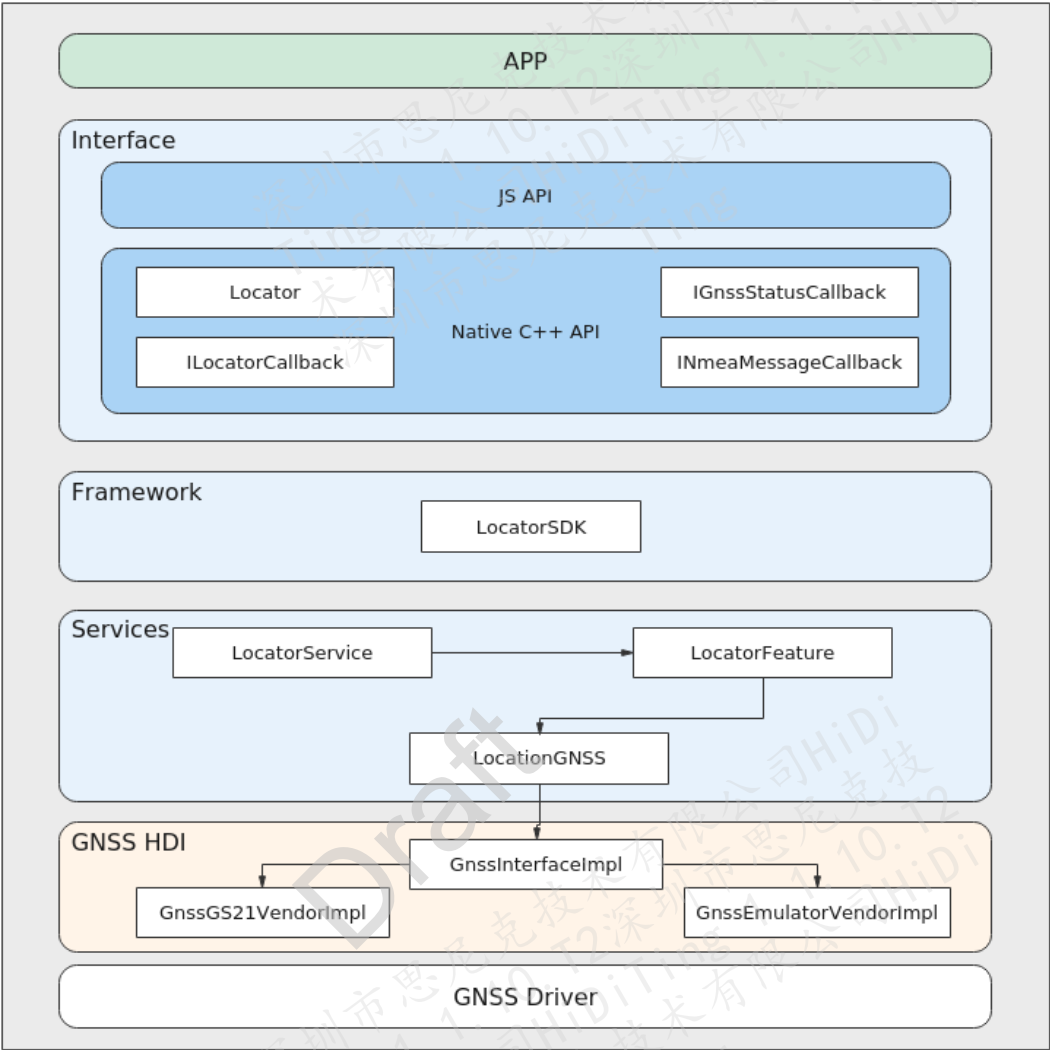
1.1 概述

位置服务（LBS，Location Based Services）又称定位服务，位置能力用于确定用户设备在哪里，系统使用位置坐标标示用户设备的位置，并使用多种定位技术提供位置服务，如 GNSS 定位、基站定位、WLAN/蓝牙定位（基站定位、WLAN/蓝牙定位统称“网络定位技术”）。通过这些定位技术，无论用户设备在室内或是户外，都可以准确地确定用户设备的位置。本组件暂时仅支持 GNSS 定位。

1.2 功能描述

定位子系统如图 1-1 所示，主要包括定位接口、定位框架、定位 HDI 等，对应用提供基本的定位管理以及定位 API 能力，构建基于轻量 OS 的定位服务框架，提供硬件资源较小的物联网设备的定位能力。

图1-1 定位子系统架构图



定位子系统分层实现，主要由 4 个层次组成：Interface（接口）、Framework（框架）、Services（服务）、GNSS HDI（硬件抽象）。

- Interface 接口层提供外部使用的 C++/JS API，负责为位置服务提供外部能力，主要提供位置服务的开启和关闭能力、位置服务的状态订阅能力、GNSS 的定位信息上报、卫星信息上报能力、NMEA 信息上报能力。
- Framework 框架层实现 Interface 接口，向下对接 Locator Service，对上提供定位能力。
- Services 服务层对接 SamgrLite，实现定位子系统的服务化，向上提供 LocatorSDK 的定位能力，管理 GNSS 定位；向下对接 GNSS 硬件抽象层，负责核心的位置服务的业务逻辑管理，如定位请求管理、位置上报管理、定位能力管理等。

- GNSS 硬件抽象层屏蔽硬件差异，负责硬件驱动对接管理，包括和 GS21 对接的 GnsGS21VendorImpl 以及模拟驱动 GnsEmulatorVendorImpl。

1.3 Native API 参考

1.3.1 Locator 类

Locator 模块主要提供获取定位服务使能状态、使能/去使能定位服务、启动/停止定位、订阅/取消订阅 GNSS 状态回调函数、订阅/取消订阅 NMEA 消息回调函数等功能。提供以下 API：

- GetInstance：获取 Locator 实例（单例模式）。
- IsLocationEnabled：查询定位服务是否使能。
- EnableAbility：使能/去使能定位服务。
- StartLocating：启动定位。
- StopLocating：停止定位。
- RegisterGnssStatusCallback：订阅 GNSS 状态回调函数。
- UnregisterGnssStatusCallback：取消订阅 GNSS 状态回调函数。
- RegisterNmeaMessageCallback：订阅 NMEA 消息回调函数。
- UnregisterNmeaMessageCallback：取消订阅 NMEA 消息回调函数。
- QuerySupportCoordinateSystemType：查询支持的坐标系。
- IsLocationPrivacyConfirmed：查询用户是否同意定位服务隐私申明，是否同意启用定位服务。
- SetLocationPrivacyConfirmStatus：设置用户确认定位服务隐私申明的状态，记录用户是否同意启用定位服务。
- SendCommand：向定位子系统发送扩展命令。

1.3.1.1 GetInstance

【描述】

获取 Locator 实例。

【语法】

```
static Locator &GetInstance();
```

【参数】

无

【返回值】

返回值	描述
Locator &	获取 Locator 实例，单例模式。

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.1.2 IsLocationEnabled

【描述】

查询定位服务是否使能。

【语法】

LocationErrCode IsLocationEnabled(bool &isEnabled);

【参数】

参数名称	描述	输入/输出
isEnabled	是否使能。	输出

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.1.3 EnableAbility

【描述】

使能定位服务。

【语法】

LocationErrCode EnableAbility(bool enable);

【参数】

参数名称	描述	输入/输出
enable	true：使能定位服务。 false：去使能定位服务。	输入

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

去使能需要在使能接口调用成功之后调用。

【举例】

具体示例内容请参见“1.7 开发指导”章节。

【相关主题】

无。

1.3.1.4 StartLocating

【描述】

启动定位服务。

【语法】

LocationErrCode StartLocating(const RequestConfig *requestConfig, ILocatorCallback *callback);

【参数】

参数名称	描述	输入/输出
requestConfig	启动定位的请求配置。	输入
callback	定位服务状态回调。	输入

【返回值】

返回值	描述
LocationErrCode	参考“1.5 错误码”章节说明。

【芯片差异】

无。

【注意】

必须在 EnableAbility 使能定位服务接口调用成功之后。

【举例】

具体示例内容请参见“1.7 开发指导”章节。

【相关主题】

无。

1.3.1.5 StopLocating

【描述】

停止定位服务。

【语法】

LocationErrCode StopLocating(ILocatorCallback *callback);

【参数】

参数名称	描述	输入/输出
callback	定位服务状态回调。	输入

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

1. 必须在 [EnableAbility](#) 使能定位服务接口调用成功之后。
2. 必须在 [StartLocating](#) 启动定位服务接口调用成功之后。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.1.6 RegisterGnssStatusCallback

【描述】

订阅 GNSS 状态回调函数。

【语法】

LocationErrCode RegisterGnssStatusCallback(IGnssStatusCallback *callback);

【参数】

参数名称	描述	输入/输出
callback	GNSS 状态回调函数。	输入

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

1. 必须在 [EnableAbility](#) 使能定位服务接口调用成功之后。
2. 必须在 [StartLocating](#) 启动定位服务之前。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.1.7 UnregisterGnssStatusCallback

【描述】

取消订阅 GNSS 状态回调函数。

【语法】

LocationErrCode UnregisterGnssStatusCallback(IGnssStatusCallback *callback);

【参数】

参数名称	描述	输入/输出
callback	GNSS 状态回调函数。	输入

【返回值】

返回值	描述
LocationErrCode	参考“1.5 错误码”章节说明。

【芯片差异】

无。

【注意】

- 1. 必须在 [EnableAbility](#) 使能定位服务接口调用成功之后。
- 2. 必须在 [RegisterGnssStatusCallback](#) 订阅 GNSS 状态回调成功之后。

【举例】

具体示例内容请参见“1.7 开发指导”章节。

【相关主题】

无。

1.3.1.8 RegisterNmeaMessageCallback

【描述】

订阅 NMEA 消息回调函数。

【语法】

LocationErrCode RegisterNmeaMessageCallback(INmeaMessageCallback *callback);

【参数】

参数名称	描述	输入/输出
callback	NMEA 信息回调函数。	输入

【返回值】

返回值	描述
LocationErrCode	参考“1.5 错误码”章节说明。

【芯片差异】

无。

【注意】

- 1. 必须在 [EnableAbility](#) 使能定位服务接口调用成功之后。
- 2. 必须在 [StartLocating](#) 启动定位服务之前。

【举例】

具体示例内容请参见“1.7 开发指导”章节。

【相关主题】

无。

1.3.1.9 UnregisterNmeaMessageCallback

【描述】

取消订阅 NMEA 消息回调函数。

【语法】

LocationErrCode UnregisterNmeaMessageCallback(INmeaMessageCallback *callback);

【参数】

参数名称	描述	输入/输出
callback	NMEA 信息回调函数。	输入

【返回值】

返回值	描述
LocationErrCode	参考“1.5 错误码”章节说明。

【芯片差异】

无。

【注意】

1. 必须在 [EnableAbility](#) 使能定位服务接口调用成功之后。
2. 必须在 [RegisterNmeaMessageCallback](#) 订阅 NMEA 消息成功之后。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.1.10 QuerySupportCoordinateSystemType

【描述】

查询支持的坐标系。

【语法】

```
LocationErrCode  
QuerySupportCoordinateSystemType(std::vector<LocationCoordinateSystemType>&  
coordinateSystemTypes);
```

【参数】

参数名称	描述	输入/输出
coordinateSystemTypes	支持的坐标系统数组。参考“ 1.4.11 LocationCoordinateSystemType ”。	输出

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

必须在 [EnableAbility](#) 使能定位服务接口调用成功之后。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.1.11 IsLocationPrivacyConfirmed

【描述】

查询用户是否同意定位服务隐私申明，是否同意启用定位服务。

【语法】

```
LocationErrCode IsLocationPrivacyConfirmed(const int type, bool &isConfirmed);
```

【参数】

参数名称	描述	输入/输出
type	隐私申明类型，暂不支持。	输出
isConfirmed	true：用户同意定位服务隐私申明。 false：用户不同意定位服务隐私申明。	输出

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

暂不支持该接口。

【举例】

无。

【相关主题】

无。

1.3.1.12 SetLocationPrivacyConfirmStatus

【描述】

设置用户确认定位服务隐私声明的状态，记录用户是否同意启用定位服务。

【语法】

LocationErrCode SetLocationPrivacyConfirmStatus(const int type, bool isConfirmed);

【参数】

参数名称	描述	输入/输出
type	隐私声明类型，暂不支持。	输入
isConfirmed	true：用户同意定位服务隐私声明。 false：用户不同意定位服务隐私声明。	输入

【返回值】

返回值	描述
LocationErrCode	参考“ 1.5 错误码 ”章节说明。

【芯片差异】

无。

【注意】

暂不支持该接口。

【举例】

无。

【相关主题】

无。

1.3.1.13 SendCommand

【描述】

向定位子系统发送扩展命令。

【语法】

LocationErrCode SendCommand(const LocationCommand *commands);

【参数】

参数名称	描述	输入/输出
commands	扩展命令参数，详情请参见“1.4.4 LocationCommand”。	输入

【返回值】

返回值	描述
LocationErrCode	参考“1.5 错误码”章节说明。

【芯片差异】

无。

【注意】

暂不支持该接口。

【举例】

无。

【相关主题】

无。

1.3.2 ILocatorCallback 类

定位服务定位结果、定位状态及定位错误信息回调函数基类。

提供以下 API：

- OnLocationReport：上报定位结果信息。
- OnLocatingStatusChange：上报定位服务状态改变。
- OnErrorReport：上报定位服务错误信息。

1.3.2.1 OnLocationReport

【描述】

上报定位结果信息。

【语法】

```
virtual void OnLocationReport(const std::shared_ptr<Location>& location) = 0;
```

【参数】

参数名称	描述	输入/输出
location	定位结果，详情请参见“ 1.4.1 Location ”。	输出

【返回值】

无

【芯片差异】

无。

【注意】

不能在 OnLocationReport 函数实现中执行耗时操作，可能导致定位结果阻塞。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.2.2 OnLocatingStatusChange

【描述】

上报定位服务状态改变。

【语法】

```
virtual void OnLocatingStatusChange(const int status) = 0;
```

【参数】

参数名称	描述	输入/输出
status	定位服务状态, 详情请参见“ 1.4.12 LocationLocatingStatus ”。	输出

【返回值】

无

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.2.3 OnErrorReport

【描述】

上报定位服务错误信息。

【语法】

```
virtual void OnErrorReport(const int errorCode) = 0;
```

【参数】

参数名称	描述	输入/输出
errorCode	定位服务错误信息。	输出

【返回值】

无

【芯片差异】

无。

【注意】

暂不支持该接口。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.3 IGnssStatusCallback 类

GNSS 卫星状态信息回调函数基类。

提供以下 API：

- OnStatusChange：上报卫星状态信息。

1.3.3.1 OnStatusChange

【描述】

上报卫星状态信息。

【语法】

```
virtual void OnStatusChange(const std::shared_ptr<SatelliteStatus>& statusInfo) = 0;
```

【参数】

参数名称	描述	输入/输出
statusInfo	卫星状态信息，详情请参见“ 1.4.3 SatelliteStatus ”。	输出

【返回值】

无

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.3.4 INmeaMessageCallback 类

NMEA 信息回调函数基类。

提供以下 API：

- OnMessageChange：上报 NMEA 信息。

1.3.4.1 OnMessageChange

【描述】

上报 NMEA 信息。

【语法】

```
virtual void OnMessageChange(int64_t timestamp, const std::string msg) = 0;
```

【参数】

参数名称	描述	输入/输出
timestamp	对应 NMEA 的时间戳信息，1970 年 1 月 1 日以来的毫秒数。	输出
msg	对应的标准 NMEA，由 GS21 芯片输出，兼容 NMEA0183 4.1 标准协议。	输出

【返回值】

无

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“[1.7 开发指导](#)”章节。

【相关主题】

无。

1.4 数据类型、数据结构

- Location：定位结果信息数据类型。
- RequestConfig：定位请求配置信息。
- SatelliteStatus：GNSS 卫星状态信息。
- LocationCommand：定位子系统扩展命令。
- LocationSourceType：定位结果的来源。
- LocationRequestScenario：位置请求中定位场景类型。
- LocationRequestPriority：位置请求中位置信息优先级类型。
- LocationSatelliteConstellation：表示卫星星座类型。
- LocationSatelliteAdditionalInfo：卫星附加信息类型。
- LocationErr：持续定位过程中的错误信息。
- LocationCoordinateSystemType：坐标系类型。
- LocationLocatingStatus：持续定位状态。

1.4.1 Location

【说明】

定位结果信息数据类型。

【定义】

```
class Location {
public:
    Location();
    explicit Location(Location &location);
    ~Location() = default;
    inline double GetLatitude() const
    {
        return latitude_;
    }
    inline void SetLatitude(double latitude)
    {
        latitude_ = latitude;
    }
    inline double GetLongitude() const
    {
        return longitude_;
    }
    inline void SetLongitude(double longitude)
    {
        longitude_ = longitude;
    }
    inline double GetAltitude() const
    {
        return altitude_;
    }
    inline void SetAltitude(double altitude)
    {
        altitude_ = altitude;
    }
    inline double GetAccuracy() const
    {
        return accuracy_;
    }
    inline void SetAccuracy(double accuracy)
    {
        accuracy_ = accuracy;
    }
    inline double GetSpeed() const
    {
        return speed_;
    }
    inline void SetSpeed(double speed)
    {
```

```

        speed_ = speed;
    }
    inline double GetDirection() const
    {
        return direction_;
    }
    inline void SetDirection(double direction)
    {
        direction_ = direction;
    }
    inline int64_t GetTimeStamp() const
    {
        return timeStamp_;
    }
    inline void SetTimeStamp(int64_t timeStamp)
    {
        timeStamp_ = timeStamp;
    }
    inline int64_t GetTimeSinceBoot() const
    {
        return timeSinceBoot_;
    }
    inline void SetTimeSinceBoot(int64_t timeStamp)
    {
        timeSinceBoot_ = timeStamp;
    }
    inline std::vector<std::string> GetAdditions() const
    {
        return additions_;
    }
    inline void SetAdditions(std::vector<std::string> additions, bool ifAppend)
    {
        if (!ifAppend) {
            std::vector<std::string>().swap(additions_);
        }
        for (auto it = additions.begin(); it != additions.end(); ++it) {
            additions_.push_back(*it);
        }
    }
    inline int64_t GetAdditionSize() const
    {
        return additionSize_;
    }

```

```
inline void SetAdditionSize(int64_t size)
{
    additionSize_ = size;
}
inline int32_t GetUncertaintyOfTimeSinceBoot() const
{
    return uncertaintyOfTimeSinceBoot_;
}
inline void SetUncertaintyOfTimeSinceBoot(int32_t uncertaintyOfTimeSinceBoot)
{
    uncertaintyOfTimeSinceBoot_ = uncertaintyOfTimeSinceBoot;
}
inline double GetAltitudeAccuracy() const
{
    return altitudeAccuracy_;
}
inline void SetAltitudeAccuracy(double altitudeAccuracy)
{
    altitudeAccuracy_ = altitudeAccuracy;
}
inline double GetSpeedAccuracy() const
{
    return speedAccuracy_;
}
inline void SetSpeedAccuracy(double speedAccuracy)
{
    speedAccuracy_ = speedAccuracy;
}
inline double GetDirectionAccuracy() const
{
    return directionAccuracy_;
}
inline void SetDirectionAccuracy(double directionAccuracy)
{
    directionAccuracy_ = directionAccuracy;
}
inline int32_t GetLocationSourceType() const
{
    return locationSourceType_;
}
inline void SetLocationSourceType(int32_t locationSourceType)
{
    locationSourceType_ = locationSourceType;
}
```

```
}
inline std::string GetUuid() const
{
    return uuid_;
}
inline void SetUuid(std::string uuid)
{
    uuid_ = uuid;
}
inline std::map<std::string, std::string> GetAdditionsMap() const
{
    return additionsMap_;
}
std::string ToString() const;
bool LocationEqual(const Location &location);
bool AdditionEqual(const Location &location);
private:
    double latitude_;
    double longitude_;
    double altitude_;
    double accuracy_;
    double speed_;
    double direction_;
    int64_t timeStamp_;
    int64_t timeSinceBoot_;
    std::vector<std::string> additions_;
    int64_t additionSize_;
    std::map<std::string, std::string> additionsMap_;
    double altitudeAccuracy_;
    double speedAccuracy_;
    double directionAccuracy_;
    int64_t uncertaintyOfTimeSinceBoot_;
    int32_t locationSourceType_;
    std::string uuid_;
};
```

【成员】

成员名称	描述
latitude_	纬度信息，正值表示北纬，负值表示南纬。取值范围为-90 ~ +90。仅支持 WGS84 坐标系。

成员名称	描述
longitude_	经度信息，正值表示东经，负值表示西经。取值范围为-180 ~ +180。仅支持 WGS84 坐标系。
altitude_	高度信息，单位：m。
accuracy_	精度信息，单位：m，暂不支持。
speed_	速度信息，单位：m/s。
direction_	航向信息。单位是“度”，取值范围为 0 ~ 360。
timeStamp_	位置时间戳，UTC 格式。
timeSinceBoot_	位置时间戳，开机时间格式。
additions_	附加信息，暂不支持。
additionSize_	附加信息数量。取值范围为 ≥ 0 ，暂不支持。
additionsMap_	附加信息。具体内容和顺序与 additions 一致，暂不支持。
altitudeAccuracy_	高度信息的精度，单位：m，暂不支持。
speedAccuracy_	速度信息的精度，单位：m/s，暂不支持。
directionAccuracy_	航向信息的精度。单位是“度”，取值范围为 0 ~ 360，暂不支持。
uncertaintyOfTimeSinceBoot_	位置时间戳的不确定度，暂不支持。
locationSourceType_	定位结果的来源，详情请参见“ 1.4.5 LocationSourceType ”，当前仅支持 GNSS_TYPE。
uuid_	Universally Unique Identifier，通用唯一识别码，暂不支持。

【注意事项】

无。

【相关数据类型及接口】

```
class ILocatorCallback {
```

```
public:
    virtual void OnLocationReport(const std::shared_ptr<Location>& location) = 0;
};
```

1.4.2 RequestConfig

【说明】

定位请求配置信息。

【定义】

```
class RequestConfig {
public:
    RequestConfig();
    explicit RequestConfig(const int scenario);
    ~RequestConfig() = default;

    inline int GetScenario() const
    {
        return scenario_;
    }
    inline void SetScenario(int scenario)
    {
        scenario_ = scenario;
    }
    inline void SetPriority(int priority)
    {
        priority_ = priority;
    }
    inline int GetPriority() const
    {
        return priority_;
    }
    inline int GetTimeInterval() const
    {
        return timeInterval_;
    }
    inline void SetTimeInterval(int timeInterval)
    {
        timeInterval_ = timeInterval;
    }
    inline double GetDistanceInterval() const
    {
        return distanceInterval_;
    }
};
```

```

}
inline void SetDistanceInterval(double distanceInterval)
{
    distanceInterval_ = distanceInterval;
}
inline float GetMaxAccuracy() const
{
    return maxAccuracy_;
}
inline void SetMaxAccuracy(float maxAccuracy)
{
    maxAccuracy_ = maxAccuracy;
}
inline void SetFixNumber(int fixNumber)
{
    fixNumber_ = fixNumber;
}
inline int GetFixNumber() const
{
    return fixNumber_;
}
inline void SetTimeOut(int time)
{
    timeOut_ = time;
}
inline int GetTimeOut() const
{
    return timeOut_;
}
inline void SetTimeStamp(int64_t time)
{
    timestamp_ = time;
}
inline int64_t GetTimeStamp() const
{
    return timestamp_;
}
std::string ToString() const;
void Set(const RequestConfig& requestConfig);
bool IsSame(RequestConfig& requestConfig);
private:
    int scenario_;
    int timeInterval_; /* Units are seconds */

```

```
double distanceInterval_ = 0.0;
float maxAccuracy_;
int fixNumber_;
int priority_;
int timeOut_;
int64_t timestamp_;
};
```

【成员】

成员名称	描述
scenario_	场景信息。当 scenario 取值为 UNSET 时, priority 参数生效, 否则 priority 参数不生效; 当 scenario 和 priority 均取值为 UNSET 时, 无法发起定位请求。取值范围请参见“ 1.4.6 LocationRequestScenario ”的定义。
timeInterval_	上报位置信息的时间间隔, 单位: s。取值范围为 ≥ 0 。默认值为对应定位模式下允许的最小时间间隔: 1s。当设置值小于最小间隔时, 以最小时间间隔生效。设置为 0 时不对时间间隔进行校验, 直接上报位置信息。
distanceInterval_	上报位置信息的距离间隔。单位: m, 默认值为 0, 取值范围为 ≥ 0 。等于 0 时对位置上报距离间隔无限制。
maxAccuracy_	请求位置信息时要求的精度值, 单位: m。该参数生效的情况下, 系统会对比 GNSS 上报的位置信息与应用的位置信息申请。当位置信息 Location 中的精度值 (accuracy) 小于等于应用要求的精度值 (maxAccuracy) 时, 位置信息会返回给应用; 否则系统将丢弃本次收到的位置信息。默认值为 0, 表示不限制位置信息的精度, 取值范围为 ≥ 0 。
fixNumber_	默认为 0, 暂不支持。
priority_	优先级信息。当 scenario 取值为 UNSET 时, priority 参数生效, 否则 priority 参数不生效; 当 scenario 和 priority 均取值为 UNSET 时, 无法发起定位请求。取值范围请参见“ 1.4.7 LocationRequestPriority ”的定义。
timeOut_	超时时间, 单位: ms, 最小为 1000ms。取值范围为 ≥ 1000 。暂不支持。

成员名称	描述
timestamp_	请求时间戳。暂不支持。

【注意事项】

无。

【相关数据类型及接口】

LocationErrCode StartLocating(const RequestConfig *requestConfig, ILocatorCallback *callback);

1.4.3 SatelliteStatus

【说明】

GNSS 卫星状态信息。

【定义】

```
class SatelliteStatus {
public:
    SatelliteStatus();
    explicit SatelliteStatus(SatelliteStatus &satelliteStatus);
    ~SatelliteStatus() = default;

    inline int GetSatellitesNumber() const
    {
        return satellitesNumber_;
    }
    inline void SetSatellitesNumber(int num)
    {
        satellitesNumber_ = num;
    }
    inline std::vector<int> GetSatellitelds() const
    {
        return satellitelds_;
    }
    inline void SetSatellitelds(std::vector<int> ids)
    {
        for (std::vector<int>::iterator it = ids.begin(); it != ids.end(); ++it) {
            satellitelds_.push_back(*it);
        }
    }
}
```

```

inline void SetSatelliteId(int id)
{
    satelliteIds_.push_back(id);
}
inline std::vector<double> GetCarrierToNoiseDensitys() const
{
    return carrierToNoiseDensitys_;
}
inline void SetCarrierToNoiseDensitys(std::vector<double> cn0)
{
    for (std::vector<double>::iterator it = cn0.begin(); it != cn0.end(); ++it) {
        carrierToNoiseDensitys_.push_back(*it);
    }
}
inline void SetCarrierToNoiseDensity(double cn0)
{
    carrierToNoiseDensitys_.push_back(cn0);
}
inline std::vector<double> GetAltitudes() const
{
    return altitudes_;
}
inline void SetAltitudes(std::vector<double> altitudes)
{
    for (std::vector<double>::iterator it = altitudes.begin(); it != altitudes.end(); ++it) {
        altitudes_.push_back(*it);
    }
}
inline void SetAltitude(double altitude)
{
    altitudes_.push_back(altitude);
}
inline std::vector<double> GetAzimuths() const
{
    return azimuths_;
}
inline void SetAzimuths(std::vector<double> azimuths)
{
    for (std::vector<double>::iterator it = azimuths.begin(); it != azimuths.end(); ++it) {
        azimuths_.push_back(*it);
    }
}
inline void SetAzimuth(double azimuth)

```

```

{
    azimuths_.push_back(azimuth);
}
inline std::vector<double> GetCarrierFrequencies() const
{
    return carrierFrequencies_;
}
inline void SetCarrierFrequencies(std::vector<double> cfs)
{
    for (std::vector<double>::iterator it = cfs.begin(); it != cfs.end(); ++it) {
        carrierFrequencies_.push_back(*it);
    }
}
inline void SetCarrierFrequency(double cf)
{
    carrierFrequencies_.push_back(cf);
}
inline std::vector<int> GetConstellationTypes() const
{
    return constellationTypes_;
}
inline void SetConstellationTypes(std::vector<int> types)
{
    for (std::vector<int>::iterator it = types.begin(); it != types.end(); ++it) {
        constellationTypes_.push_back(*it);
    }
}
inline void SetConstellationType(int type)
{
    constellationTypes_.push_back(type);
}
inline std::vector<int> GetSatelliteAdditionalInfoList()
{
    return additionalInfoList_;
}
inline void SetSatelliteAdditionalInfo(int additionalInfo)
{
    additionalInfoList_.push_back(additionalInfo);
}
inline void SetSatelliteAdditionalInfoList(std::vector<int> additionalInfo)
{
    for (std::vector<int>::iterator it = additionalInfo.begin();
        it != additionalInfo.end(); ++it) {

```

```
        additionalInfoList_.push_back(*it);
    }
}
private:
    int satellitesNumber_;
    std::vector<int> satellitelds_;
    std::vector<double> carrierToNoiseDensitys_;
    std::vector<double> altitudes_;
    std::vector<double> azimuths_;
    std::vector<double> carrierFrequencies_;
    std::vector<int> constellationTypes_;
    std::vector<int> additionalInfoList_;
};
```

【成员】

成员名称	描述
satellitesNumber_	卫星个数。取值范围为 ≥ 0 。
satellitelds_	每个卫星的 ID，数组类型。取值范围为 ≥ 0 。
carrierToNoiseDensitys_	载波噪声功率谱密度比，即 $cn0$ 。取值范围为 > 0 ，暂不支持。
altitudes_	卫星高度角信息。单位是“度”，取值范围为 $-90 \sim +90$ 。
azimuths_	方位角。单位是“度”，取值范围为 $0 \sim 360$ 。
carrierFrequencies_	载波频率。单位：Hz，取值范围为 ≥ 0 。
constellationTypes_	卫星星座类型，具体值参考“ 1.4.8 LocationSatelliteConstellation ”。
additionalInfoList_	卫星的附加信息。每个比特位代表不同含义，具体定义参见“ 1.4.9 LocationSatelliteAdditionalInfo ”，暂不支持。

【注意事项】

无。

【相关数据类型及接口】


```
class IGnssStatusCallback {  
public:  
    virtual void OnStatusChange(const std::shared_ptr<SatelliteStatus>& statusInfo) = 0;  
};
```

1.4.4 LocationCommand

【说明】

定位子系统扩展命令。

【定义】

```
typedef struct {  
    int scenario;  
    std::string command;  
} LocationCommand;
```

【成员】

成员名称	描述
scenario	定位场景，具体值参考“ 1.4.6 LocationRequestScenario ”。
command	扩展命令字符串。

【注意事项】

无。

【相关数据类型及接口】

```
LocationErrCode SendCommand(const LocationCommand *commands);
```

1.4.5 LocationSourceType

【说明】

定位结果的来源。

【定义】

```
enum LocationSourceType {  
    GNSS_TYPE = 1,  
    NETWORK_TYPE = 2,  
    INDOOR_TYPE = 3,
```

```
RTK_TYPE = 4,
};
```

【成员】

成员名称	描述
GNSS_TYPE	定位结果来自于 GNSS 定位技术。
NETWORK_TYPE	定位结果来自于网络定位技术。暂不支持。
INDOOR_TYPE	定位结果来自于室内高精度定位技术。暂不支持。
RTK_TYPE	定位结果来自于室外高精度定位技术。暂不支持。

【注意事项】

无。

【相关数据类型及接口】

- [Location](#)

```
inline int32_t GetLocationSourceType() const;
inline void SetLocationSourceType(int32_t locationSourceType)
```

1.4.6 LocationRequestScenario

【说明】

位置请求中定位场景类型。

【定义】

```
enum LocationRequestScenario {
    SCENE_UNSET = 0x0300,
    SCENE_NAVIGATION = 0x0301,
    SCENE_TRAJECTORY_TRACKING = 0x0302,
    SCENE_CAR_HAILING = 0x0303,
    SCENE_DAILY_LIFE_SERVICE = 0x0304,
    SCENE_NO_POWER = 0x0305
};
```

【成员】

成员名称	描述
------	----

成员名称	描述
SCENE_UNSET	未设置场景信息。表示 LocationRequestScenario 字段无效。
SCENE_NAVIGATION	导航场景。适用于在户外获取设备实时位置的场景，如车载、步行导航。主要使用 GNSS 定位技术提供定位服务，功耗较高。
SCENE_TRAJECTORY_TRACKING	运动轨迹记录场景。适用于记录用户位置轨迹的场景，如运动类应用记录轨迹功能。主要使用 GNSS 定位技术提供定位服务，功耗较高。
SCENE_CAR_HAILING	打车场景。适用于用户出行打车时定位当前位置的场景，如网约车类应用。主要使用 GNSS 定位技术提供定位服务，功耗较高。
SCENE_DAILY_LIFE_SERVICE	日常服务使用场景。适用于不需要定位用户精确位置的使用场景，如新闻资讯、网购、点餐类应用。该场景仅使用网络定位技术提供定位服务，功耗较低，暂不支持。
SCENE_NO_POWER	无功耗场景，这种场景下不会主动触发定位，会在其他应用定位时，才给当前应用返回位置，暂不支持。

【注意事项】

无。

【相关数据类型及接口】

- [RequestConfig](#)

```
inline int GetScenario() const
inline void SetScenario(int scenario)
```

1.4.7 LocationRequestPriority

【说明】

位置请求中位置信息优先级类型。

【定义】

```
enum LocationRequestPriority {
```

```
PRIORITY_UNSET = 0x0200,  
PRIORITY_ACCURACY = 0x0201,  
PRIORITY_LOW_POWER = 0x0202,  
PRIORITY_FAST_FIRST_FIX = 0x0203  
};
```

【成员】

成员名称	描述
PRIORITY_UNSET	未设置优先级，表示 LocationRequestPriority 无效。
PRIORITY_ACCURACY	精度优先。定位精度优先策略主要以 GNSS 定位技术为主。
PRIORITY_LOW_POWER	低功耗优先，暂不支持。
PRIORITY_FAST_FIRST_FIX	快速获取位置优先，如果应用希望快速拿到一个位置，可以将优先级设置为该字段，暂不支持。

【注意事项】

无。

【相关数据类型及接口】

- [RequestConfig](#)

```
inline void SetPriority(int priority);  
inline int GetPriority() const;
```

1.4.8 LocationSatelliteConstellation

【说明】

表示卫星星座类型。

【定义】

```
enum LocationSatelliteConstellation {  
    SV_CONSTELLATION_CATEGORY_UNKNOWN = 0,  
    SV_CONSTELLATION_CATEGORY_GPS,  
    SV_CONSTELLATION_CATEGORY_GLONASS,  
    SV_CONSTELLATION_CATEGORY_QZSS,  
    SV_CONSTELLATION_CATEGORY_BEIDOU,  
    SV_CONSTELLATION_CATEGORY_GALILEO,  
};
```

};

【成员】

成员名称	描述
SV_CONSTELLATION_CATEGORY_UNKNOWN	默认值。
SV_CONSTELLATION_CATEGORY_GPS	GPS (Global Positioning System) , 即全球定位系统, 是美国研制发射的一种以人造地球卫星为基础的高精度无线电导航的定位系统。
SV_CONSTELLATION_CATEGORY_GLOASS	GLONASS (GLOBAL NAVIGATION SATELLITE SYSTEM) , 是苏联/俄罗斯研制卫星导航系统。
SV_CONSTELLATION_CATEGORY_QZSS	QZSS (Quasi-Zenith Satellite System) , 即准天顶卫星系统, 是以三颗人造卫星透过时间转移完成全球定位系统区域性功能的卫星扩增系统, 是日本研发的卫星系统。
SV_CONSTELLATION_CATEGORY_BEIDOU	北斗卫星导航系统 (Beidou Navigation Satellite System) 是中国自行研制的全球卫星导航系统。
SV_CONSTELLATION_CATEGORY_GALILEO	GALILEO (Galileo satellite navigation system) , 即伽利略卫星导航系统, 是由欧盟研制和建立的全球卫星导航定位系统。

【注意事项】

无。

【相关数据类型及接口】

- [SatelliteStatus](#)
- [IGnssStatusCallback](#)

```
class IGnssStatusCallback {
public:
    virtual void OnStatusChange(const std::shared_ptr<SatelliteStatus>& statusInfo) = 0;
};
```

1.4.9 LocationSatelliteAdditionalInfo

【说明】

卫星附加信息类型。

【定义】

```
enum LocationSatelliteAdditionalInfo {  
    SV_ADDITIONAL_INFO_NULL = 0,  
    SV_ADDITIONAL_INFO_EPHEMERIS_DATA_EXIST = 1,  
    SV_ADDITIONAL_INFO_ALMANAC_DATA_EXIST = 2,  
    SV_ADDITIONAL_INFO_USED_IN_FIX = 4,  
    SV_ADDITIONAL_INFO_CARRIER_FREQUENCY_EXIST = 8,  
};
```

【成员】

成员名称	描述
SV_ADDITIONAL_INFO_NULL	默认值。
SV_ADDITIONAL_INFO_EPHEMERIS_DATA_EXIST	本卫星具有星历数据，暂不支持。
SV_ADDITIONAL_INFO_ALMANAC_DATA_EXIST	本卫星具有年历数据，暂不支持。
SV_ADDITIONAL_INFO_USED_IN_FIX	在最新的位置解算中使用了本卫星，暂不支持。
SV_ADDITIONAL_INFO_CARRIER_FREQUENCY_EXIST	本卫星具有载波频率，暂不支持。

【注意事项】

无。

【相关数据类型及接口】

- [SatelliteStatus](#)
- [IGnssStatusCallback](#)

```
class IGnssStatusCallback {  
public:  
    virtual void OnStatusChange(const std::shared_ptr<SatelliteStatus>& statusInfo) = 0;  
};
```

1.4.10 LocationErr

【说明】

持续定位过程中的错误信息。

【定义】

```
enum LocationErr {
    LOCATING_FAILED_DEFAULT = -1,
    LOCATING_FAILED_LOCATION_PERMISSION_DENIED = -2,
    LOCATING_FAILED_BACKGROUND_PERMISSION_DENIED = -3,
    LOCATING_FAILED_LOCATION_SWITCH_OFF = -4,
};
```

【成员】

成员名称	描述
LOCATING_FAILED_DEFAULT	默认值。
LOCATING_FAILED_LOCATION_PERMISSION_DENIED	权限校验失败导致持续定位失败，暂不支持。
LOCATING_FAILED_BACKGROUND_PERMISSION_DENIED	应用在后台时位置权限校验失败导致持续定位失败，暂不支持。
LOCATING_FAILED_LOCATION_SWITCH_OFF	位置信息开关关闭导致持续定位失败。

【注意事项】

无。

【相关数据类型及接口】

```
virtual void OnErrorReport(const int errorCode) = 0;
```

1.4.11 LocationCoordinateSystemType

【说明】

坐标系类型。

【定义】

```
enum LocationCoordinateSystemType {
    WGS84 = 1,
```

```
GCJ02,
};
```

【成员】

成员名称	描述
WGS84	World Geodetic System 1984，是为 GPS 全球定位系统使用而建立的坐标系统。
GCJ02	GCJ-02 是由中国国家测绘局制订的地理信息系统的坐标系统。

【注意事项】

无。

【相关数据类型及接口】

```
LocationErrCode SendCommand(const LocationCommand *commands);
```

1.4.12 LocationLocatingStatus

【说明】

持续定位状态。

【定义】

```
enum LocationLocatingStatus {
    LOCATING_STARTED = 0x0002,
    LOCATING_STOPED = 0x0003,
};
```

【成员】

成员名称	描述
LOCATING_STARTED	持续定位启动。
LOCATING_STOPED	持续定位停止。

【注意事项】

无。

【相关数据类型及接口】

```
class ILocatorCallback {
public:
    enum {
        RECEIVE_LOCATION_STATUS_EVENT = 1,
        RECEIVE_ERROR_INFO_EVENT = 2,
        RECEIVE_LOCATION_INFO_EVENT = 3,
    };
    virtual void OnLocationReport(const std::shared_ptr<Location>& location) = 0;
    virtual void OnLocatingStatusChange(const int status) = 0;
    virtual void OnErrorReport(const int errorCode) = 0;
};
```

1.5 错误码

表1-1 定位子系统 API 错误码

错误代码	宏定义	描述
0	ERRCODE_SUCCESS	成功。
-1	ERRCODE_FAILURE	失败。
401	ERRCODE_INVALID_PARAMS	失败。
801	ERRCODE_NOT_SUPPORTED	不支持的操作。
3301000	ERRCODE_SERVICE_UNAVAILABLE	位置服务不可用。
3301200	ERRCODE_LOCATING_FAIL	定位失败，未获取到定位结果。
3301700	ERRCODE_NO_RESPONSE	请求无响应。

1.6 GNSS 套片对接说明

Location 组件仅支持 GS21 套片对接，BrandyK100 与 GS21 套片对接请参考 GS21 套片相关开发指导文档自行完成，不在本文档范围。

1.6.1 GNSS Emulator 说明

由于 BrandyK100 上没有默认对接 GS21 套片，硬件适配层提供模拟驱动实现 GnssEmulatorVendorImpl，支持在未对接 GS21 的情况下测试定位子系统。

GNSS Emulator 通过读取提前准备的保存自 GS21 实际定位结果的 NMEA 标准消息文件，解析并每隔 1s 上报文件中读取的定位结果。

默认文件位置为 “/user/gnss_nmea.log”，需要修改替换可以修改如下代码：

```
openharmy\drivers\peripheral\location\gnss\gnss_emulator\source\gnss_emulator_vendor_impl.c
pp
std::string g_gnssNmeaFile = "/user/gnss_nmea.log";
```

1.6.2 GNSS 套片使用说明

BrandyK100 上默认使用 GNSS Emulator，对已经完成适配对接 GS21 情况下，可以切换到 GnssGS21VendorImpl 使用。

1.6.2.1 编译切换

- 编译配置组件切换

修改路径：build\config\target_config\common_config.py

在'ohos_set'中将 location_gnss_emulator 修改为 location_gnss_hdi。

- 编译 CMakeLists 切换

修改路径：openharmy\drivers\peripheral\location\CMakeLists.txt

将如下代码：

```
#add_subdirectory_with_alias_if_exist(gnss/gnss_gs21
${PROJECT_BINARY_DIR}/ohos/location/location_gnss_gs21)
add_subdirectory_with_alias_if_exist(gnss/gnss_emulator
${PROJECT_BINARY_DIR}/ohos/location/location_gnss_emulator)
```

修改为：

```
add_subdirectory_with_alias_if_exist(gnss/gnss_gs21
${PROJECT_BINARY_DIR}/ohos/location/location_gnss_gs21)
#add_subdirectory_with_alias_if_exist(gnss/gnss_emulator
${PROJECT_BINARY_DIR}/ohos/location/location_gnss_emulator)
```

1.6.2.2 PGNSS 注入说明

GS21 接收机在无任何辅助信息的情况下，冷启动定位受限于自主解调广播星历耗时，一般需要 30s。

PGNSS 服务提供联网一次下载 PGNSS 数据，后续可以离线预测未来的星历数据功能，可免除自主解星历时间，缩短首次定位时间；预测时间较长，可提供 1~28 天预测数据，但预测天数越长数据下载量越大，且离线预测的精度会随时间增长变差。

定位子系统对接的 GnssGS21VendorImpl 使用 PGNSS 方案。

PGNSS 的下载和处理请参考 GS21 配套的《HiDiTingV100 XGNSS 开发指南》文档。

解析完的 PGNSS 数据需要上传到板端 “/user/xgnss” 路径。

启动定位业务之前需要使用 AT 命令设置当前系统时间，例如：“AT^TIMESSET=2024-9-11 0:41:20”。

1.7 开发指导

下面 sample 描述了定位子系统的使能、注册回调函数、启动定位到停止定位退出的过程。

对应 AT 命令：

使能定位：AT^OHOSFWK_LOCATION=enable

查询支持坐标系：AT^OHOSFWK_LOCATION=query

启动定位：AT^OHOSFWK_LOCATION=start

停止定位：AT^OHOSFWK_LOCATION=stop

去使能定位：AT^OHOSFWK_LOCATION=disable

1.7.1 定位准备工作

```
/*
*****
获取定位实例、检查定位是否已经使能
*****
*/
/* 获取AudioManager实例 */
if (g_locationContext.locator == nullptr) {
    g_locationContext.locator = &Locator::GetInstance();
}
```

```

}
if (g_locationContext.locator == nullptr) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator is nullptr\n");
    return ERRCODE_FAILURE;
}
/* 检查定位是否已经使能 */
bool isEnabled = false;
LocationErrCode errCode = g_locationContext.locator->IsLocationEnabled(isEnabled);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator IsLocationEnabled
failed:%d\n", errCode);
    return errCode;
}
if (isEnabled) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator IsLocationEnabled\n");
    return ERRCODE_SUCCESS;
}
/* 使能定位 */
errCode = g_locationContext.locator->EnableAbility(true);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator EnableAbility failed:%d\n",
errCode);
    return errCode;
}
WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "Enable Locating end\n");
return ERRCODE_SUCCESS;

```

1.7.2 查询支持坐标系

```

/*****/
查询支持坐标系
/*****/
std::vector<LocationCoordinateSystemType> coordinateSystemTypes;
LocationErrCode errCode = g_locationContext.locator-
>QuerySupportCoordinateSystemType(coordinateSystemTypes);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP,
"QuerySupportCoordinateSystemType failed:%d\n", errCode);
}
WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "query size:%lu\n",
coordinateSystemTypes.size());
for (auto type : coordinateSystemTypes) {

```

```
WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "query type:%d\n", type);
}
```

1.7.3 实现 GnssStatus 及订阅回调函数

```

/*****/
订阅GnssStatus回调函数
/*****/
/* 实现回调函数 */
class GnssStatusCallback : public IGnssStatusCallback {
public:
    GnssStatusCallback() {}
    ~GnssStatusCallback() {}
    void OnStatusChange(const std::shared_ptr<SatelliteStatus> &statusInfo) override
    {
        WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "OnStatusChange, num:%d\n",
statusInfo->GetSatellitesNumber());
        for (int32_t satIdx = 0; satIdx < statusInfo->GetSatellitesNumber(); satIdx++) {
            WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "sat index:%d svid:%d
ele:%.3f az:%.3f  cnr:%.3f \n", satIdx,
                        statusInfo->GetSatelliteIds()[satIdx], statusInfo->GetAltitudes()[satIdx],
                        statusInfo->GetAzimuths()[satIdx], statusInfo-
>GetCarrierToNoiseDensitys()[satIdx]);
        }
    }
private:
};
/* 订阅回调函数 */
GnssStatusCallback gnssStatusCallback;
LocationErrCode errCode = g_locationContext.locator-
>RegisterGnssStatusCallback(&gnssStatusCallback);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator RegisterGnssStatusCallback
failed:%d\n", errCode);
    return errCode;
}

```

1.7.4 实现 NmeaMessage 及订阅回调函数

```

/*****/
订阅NmeaMessage回调函数

```

```

/*****
/* 实现回调函数 */

class NmeaMessageCallback : public INmeaMessageCallback {
public:
    NmeaMessageCallback() {}
    ~NmeaMessageCallback() {}
    void OnMessageChange(int64_t timestamp, const std::string msg) override
    {
        WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "OnMessageChange
timestamp :%lld msg:%s\n", timestamp, msg.c_str());
    }
private:
};

/* 订阅回调函数 */

NmeaMessageCallback nmeaMessageCallback;
LocationErrCode errCode = g_locationContext.locator-
>RegisterNmeaMessageCallback(&nmeaMessageCallback);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator
RegisterNmeaMessageCallback failed:%d\n", errCode);
    return errCode;
}

```

1.7.5 实现定位回调函数及启动定位

```

/*****
注册LocatorCallback 回调函数

/*****
/* 实现回调函数 */

class LocatorCallback : public ILocatorCallback {
public:
    LocatorCallback() {}
    ~LocatorCallback() {}
    void OnLocationReport(const std::shared_ptr<Location> &location) override
    {
        WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "OnLocationReport Lat:%f \n",
location->GetLatitude());

        // 使用定位结果
    }
    void OnErrorReport(const int errorCode) override
    {

```

```

WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "error code:%d\n", errorCode);

// 处理定位错误

}

void OnLocatingStatusChange(const int status) override
{
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "OnLocatingStatusChange
status:%d\n", status);

    // 处理定位状态改变

}

private:
};

/* 启动定位 */

RequestConfig requestConfig;
requestConfig.SetPriority(PRIORITY_ACCURACY);
LocationErrCode errCode = g_locationContext.locator->StartLocating(&requestConfig,
&locatorCallback);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator StartLocating failed:%d\n",
errCode);
    return errCode;
}

```

1.7.6 取消订阅 GnssStatus 回调函数

```

/*****/

取消订阅GnssStatus回调函数

/*****/

LocationErrCode errCode = g_locationContext.locator-
>UnregisterGnssStatusCallback(&gnssStatusCallback);
if (errCode != ERRCODE_SUCCESS) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator RegisterGnssStatusCallback
failed:%d\n", errCode);
}

```

1.7.7 取消订阅 NmeaMessage 回调函数

```

/*****/

取消订阅NmeaMessage回调函数

/*****/

LocationErrCode errCode =
g_locationContext.locator->UnregisterNmeaMessageCallback(&nmeaMessageCallback);

```

```
if (errCode != ERRCODE_SUCCESS) {  
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator  
UnregisterNmeaMessageCallback failed:%d\\n", errCode);  
}
```

1.7.8 停止定位及退出

```
/*  
*****  
定位停止及退出  
*****  
*/  
  
/* 停止定位 */  
  
LocationErrCode errCode = g_locationContext.locator->  
>StopLocating(&g_locationContext.locatorCallback);  
if (errCode != ERRCODE_SUCCESS) {  
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator StopLocating failed:%d\\n",  
errCode);  
};  
  
/* 去使能定位 */  
  
errCode = g_locationContext.locator->EnableAbility(false);  
if (errCode != ERRCODE_SUCCESS) {  
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "locator EnableAbility failed:%d\\n",  
errCode);  
    return errCode;  
};
```


2 Sensor 组件

- 2.1 概述
- 2.2 功能描述
- 2.3 Native API 参考
- 2.4 数据类型、数据结构
- 2.5 错误码
- 2.6 Sensor 驱动对接说明
- 2.7 开发指导

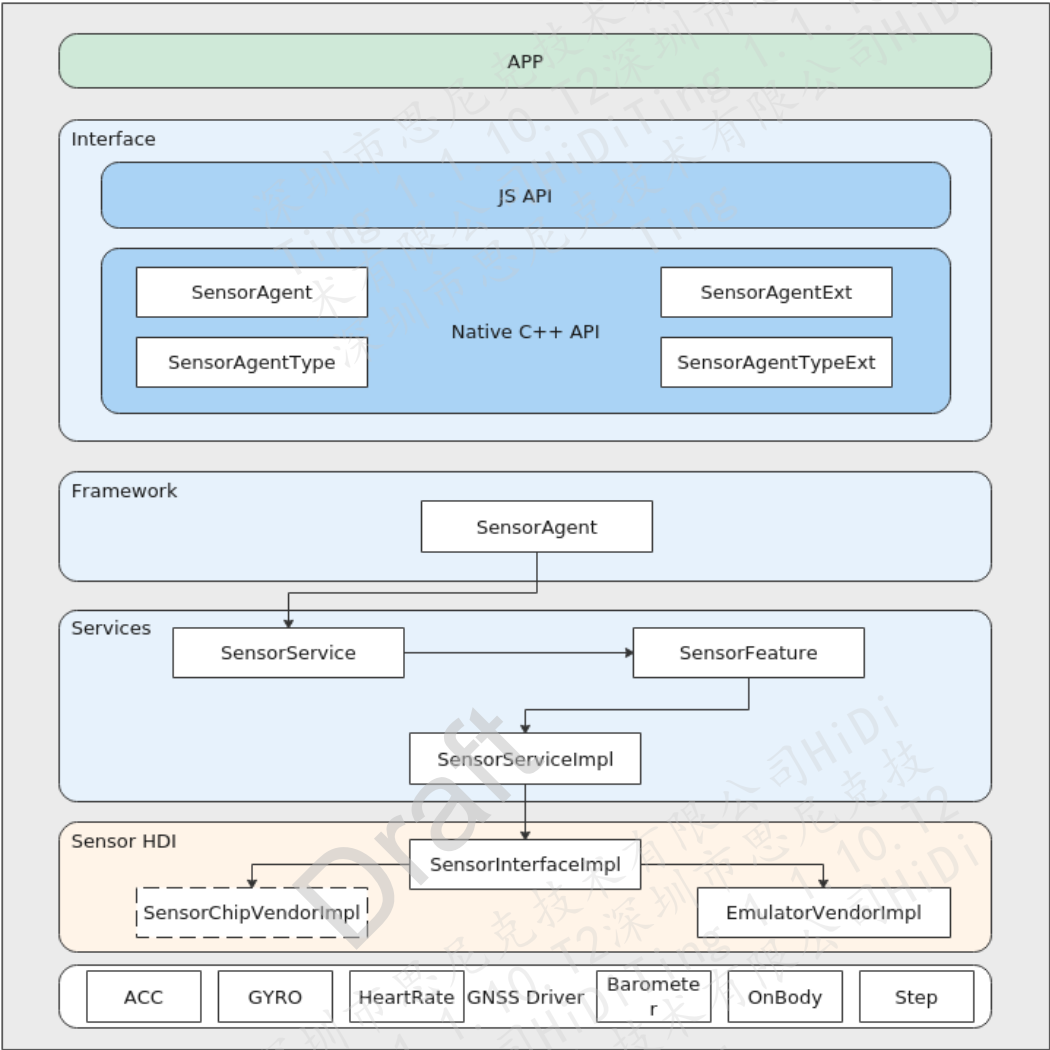
2.1 概述

Sensor 组件提供统一的轻量级 sensor 服务基础框架，支持 L0 轻量鸿蒙设备，使传感器可通过 API 开放能力。

2.2 功能描述

传感器子系统如图 2-1 所示，主要包括北向 API 接口、传感器服务框架、传感器 HDI 等，对应用提供基本的传感器管理以及传感器 API 能力，构建基于轻量 OS 的定位服务框架，提供硬件资源较小的物联网设备的传感器管理能力。

图2-1 传感器子系统架构图



传感器子系统分层实现，主要由 4 个层次组成：Interface（接口）、Framework（框架）、Services（服务）、Sensor HDI（硬件抽象）。

- Interface 接口层提供外部使用的 C++/JS API，负责为传感器服务提供外部能力，主要提供传感器服务的以下功能：
 - Sensor 列表查询
 - Sensor 启停
 - Sensor 订阅/去订阅
 - 设置数据上报模式
 - 设置采样间隔等

- Framework 框架层实现 Interface 接口。向下对接 SensorService，对上提供传感器管理能力。
- Services 服务层对接 SamgrLite，实现传感器子系统的服务化。向下对接 Sensor 硬件抽象层，负责核心的传感器服务的业务逻辑管理。
- Sensor 硬件抽象层屏蔽硬件差异，负责硬件驱动对接管理，包括和实际 Sensor 驱动对接的 SensorChipVendorImpl（暂未支持）以及模拟 Sensor 驱动实现 EmulatorVendorImpl。

2.3 Native API 参考

提供以下 API：

- GetAllSensors：获取系统中所有传感器的信息。
- SubscribeSensor：订阅传感器数据，系统会将获取到的传感器数据上报给订阅者。
- UnsubscribeSensor：去订阅传感器数据，系统将取消传感器数据上报给订阅者。
- SetBatch：设置传感器的数据采样间隔和数据上报间隔。
- ActivateSensor：使能一个传感器订阅用户，只有在传感器使能之后，订阅该传感器的用户才能获取到数据。
- DeactivateSensor：去使能一个传感器订阅用户。
- SetMode：设置传感器的数据上报模式。

2.3.1 GetAllSensors

【描述】

获取系统中所有传感器的信息。

【语法】

```
int32_t GetAllSensors(SensorInfo **sensorInfo, int32_t *count);
```

【参数】

参数名称	描述	输入/输出
sensorInfo	传感器信息，详情请参见“ 2.4.2 SensorInfo ”。	输出
count	传感器个数。	输出

【返回值】

返回值	描述
int32_t	0：成功。 - 非 0：失败，请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“2.7.1 获取设备支持的 Sensor 列表”章节。

【相关主题】

无。

2.3.2 SubscribeSensor

【描述】

订阅传感器数据，系统会将获取到的传感器数据上报给订阅者。

【语法】

int32_t SubscribeSensor(int32_t sensorTypeId, SensorUser *user);

【参数】

参数名称	描述	输入/输出
sensorTypeId	传感器类型，详情请参见“2.4.1 SensorTypeId”。	输入
user	传感器用户信息，详情请参见“2.4.6 SensorUser”。	输入

【返回值】

返回值	描述
int32_t	0：成功。 - 非 0：失败，请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“2.7.5 订阅传感器数据”章节。

【相关主题】

无。

2.3.3 UnsubscribeSensor

【描述】

去订阅传感器数据，系统将取消传感器数据上报给订阅者。

【语法】

int32_t UnsubscribeSensor(int32_t sensorTypeId, SensorUser *user);

【参数】

参数名称	描述	输入/输出
sensorTypeId	传感器类型，详情请参见“2.4.1 SensorTypeId”。	输入
user	传感器用户信息，详情请参见“2.4.6 SensorUser”。	输入

【返回值】

返回值	描述
int32_t	0：成功。 - 非 0：失败，请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

必须在“[2.3.2 SubscribeSensor](#)”成功之后调用。

【举例】

具体示例内容请参见“[2.7.6 取消传感器数据订阅](#)”章节。

【相关主题】

无。

2.3.4 SetBatch

【描述】

设置传感器的数据采样间隔和数据上报间隔。

【语法】

int32_t SetBatch(int32_t sensorTypeId, SensorUser *user, int64_t samplingInterval, int64_t reportInterval);

【参数】

参数名称	描述	输入/输出
sensorTypeId	传感器类型，详情请参见“ 2.4.1 SensorTypeId ”。	输入
user	传感器用户信息，详情请参见“ 2.4.6 SensorUser ”。	输入
samplingInterval	传感器采样间隔，单位：ns。	输入
reportInterval	传感器数据上报间隔，单位：ns。	输入

【返回值】

返回值	描述
int32_t	0：成功。 - 非 0：失败，请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

暂不支持该接口。

【举例】

无。

【相关主题】

无。

2.3.5 ActivateSensor

【描述】

使能一个传感器订阅用户，只有在传感器使能之后，订阅该传感器的用户才能获取到数据。

【语法】

```
int32_t ActivateSensor(int32_t sensorTypeId, SensorUser *user);
```

【参数】

参数名称	描述	输入/输出
sensorTypeId	传感器类型，详情请参见“2.4.1 SensorTypeId”。	输入
user	传感器用户信息，详情请参见“2.4.6 SensorUser”。	输入

【返回值】

返回值	描述
int32_t	0：成功。 - 非 0：失败，请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

【举例】

具体示例内容请参见“2.7.4 使能传感器”章节。

【相关主题】

无。

2.3.6 DeactivateSensor

【描述】

去订阅传感器数据，系统将取消传感器数据上报给订阅者。

【语法】

int32_t DeactivateSensor(int32_t sensorTypeId, SensorUser *user);

【参数】

参数名称	描述	输入/输出
sensorTypeId	传感器类型，详情请参见“2.4.1 SensorTypeId”。	输入
user	传感器用户信息，详情请参见“2.4.6 SensorUser”。	输入

【返回值】

返回值	描述
int32_t	0: 成功。 - 非 0: 失败, 请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

必须在“[2.3.5 ActivateSensor](#)”成功之后调用。

【举例】

具体示例内容请参见“[2.7.7 去使能一个传感器](#)”章节。

【相关主题】

无。

2.3.7 SetMode

【描述】

设置传感器的数据上报模式。

【语法】

```
int32_t SetMode(int32_t sensorTypeId, SensorUser *user, int32_t mode);
```

【参数】

参数名称	描述	输入/输出
sensorTypeId	传感器类型, 详情请参见“ 2.4.1 SensorTypeId ”。	输入
user	传感器用户信息, 详情请参见“ 2.4.6 SensorUser ”。	输入
mode	传感器数据上报模式, 详情请参见“ 2.4.7 SensorMode ”。	输入

【返回值】

返回值	描述
int32_t	0：成功。 - 非 0：失败，请参考“2.5 错误码”章节说明。

【芯片差异】

无。

【注意】

暂不支持该接口。

【举例】

无。

【相关主题】

无。

2.4 数据类型、数据结构

提供以下数据结构：

- SensorTypeId：传感器类型标识。
- SensorInfo：设备支持的传感器信息。
- SensorEvent：传感器上报数据的结构。
- RecordSensorCallback：传感器管理服务的传感器上报数据回调函数的定义。
- UserData：传感器订阅的预留字段。
- SensorUser：传感器的订阅用户。
- SensorMode：传感器的打开模式。
- AccelData：加速度传感器的数据结构。
- GyroData：角速度传感器的数据结构。
- PressureData：气压传感器的数据结构。
- SENSOR_NAME_MAX_LEN：Sensor 命名的最大长度。

- SENSOR_USER_DATA_SIZE：传感器用户数据的大小。
- VERSION_MAX_LEN：传感器版本号最大长度。

2.4.1 SensorTypeId

【说明】

传感器类型标识。

【定义】

```
typedef enum SensorTypeId {
    SENSOR_TYPE_ID_NONE = 0,                /**< None */
    SENSOR_TYPE_ID_ACCELEROMETER = 1,        /**< Acceleration sensor */
    SENSOR_TYPE_ID_GYROSCOPE = 2,            /**< Gyroscope sensor */
    SENSOR_TYPE_ID_PHOTOPLETHYSMOGRAPH = 3,  /**< Photoplethysmography
sensor */
    SENSOR_TYPE_ID_ELECTROCARDIOGRAPH = 4,   /**< Electrocardiogram (ECG)
sensor */
    SENSOR_TYPE_ID_AMBIENT_LIGHT = 5,        /**< Ambient light sensor */
    SENSOR_TYPE_ID_MAGNETIC_FIELD = 6,        /**< Magnetic field sensor */
    SENSOR_TYPE_ID_CAPACITIVE = 7,           /**< Capacitive sensor */
    SENSOR_TYPE_ID_BAROMETER = 8,            /**< Barometric pressure sensor */
    SENSOR_TYPE_ID_TEMPERATURE = 9,          /**< Temperature sensor */
    SENSOR_TYPE_ID_HALL = 10,                /**< Hall effect sensor */
    SENSOR_TYPE_ID_GESTURE = 11,             /**< Gesture sensor */
    SENSOR_TYPE_ID_PROXIMITY = 12,           /**< Proximity sensor */
    SENSOR_TYPE_ID_HUMIDITY = 13,            /**< Humidity sensor */
    SENSOR_TYPE_ID_PHYSICAL_MAX = 0xFF,      /**< Maximum type ID of a physical
sensor */
    SENSOR_TYPE_ID_ORIENTATION = 256,        /**< Orientation sensor */
    SENSOR_TYPE_ID_GRAVITY = 257,            /**< Gravity sensor */
    SENSOR_TYPE_ID_LINEAR_ACCELERATION = 258, /**< Linear acceleration sensor */
    SENSOR_TYPE_ID_ROTATION_VECTOR = 259,    /**< Rotation vector sensor */
    SENSOR_TYPE_ID_AMBIENT_TEMPERATURE = 260, /**< Ambient temperature sensor */
    SENSOR_TYPE_ID_MAGNETIC_FIELD_UNCALIBRATED = 261, /**< Uncalibrated
magnetic field sensor */
    SENSOR_TYPE_ID_GAME_ROTATION_VECTOR = 262, /**< Game rotation vector
sensor */
    SENSOR_TYPE_ID_GYROSCOPE_UNCALIBRATED = 263, /**< Uncalibrated gyroscope
sensor */
    SENSOR_TYPE_ID_SIGNIFICANT_MOTION = 264, /**< Significant motion sensor */
    SENSOR_TYPE_ID_PEDOMETER_DETECTION = 265, /**< Pedometer detection sensor
*/
}
```

```
SENSOR_TYPE_ID_PEDOMETER = 266,          /**< Pedometer sensor */
SENSOR_TYPE_ID_GEOMAGNETIC_ROTATION_VECTOR = 277, /**< Geomagnetic
rotation vector sensor */
SENSOR_TYPE_ID_HEART_RATE = 278,          /**< Heart rate sensor */
SENSOR_TYPE_ID_DEVICE_ORIENTATION = 279,  /**< Device orientation sensor */
SENSOR_TYPE_ID_WEAR_DETECTION = 280,      /**< Wear detection sensor */
SENSOR_TYPE_ID_ACCELEROMETER_UNCALIBRATED = 281, /**< Uncalibrated
acceleration sensor */
SENSOR_TYPE_ID_MAX = 0xFFFF,             /**< Maximum number of sensor type IDs*/
} SensorTypeId;
```

【成员】

成员名称	描述
SENSOR_TYPE_ID_NONE	传感器类型 ID 起始值。
SENSOR_TYPE_ID_ACCELEROMETER	加速度传感器。
SENSOR_TYPE_ID_GYROSCOPE	陀螺仪传感器。
SENSOR_TYPE_ID_PHOTOPLETHYSMOGRAPH	心率传感器。
SENSOR_TYPE_ID_ELECTROCARDIOGRAPH	心电（ECG）传感器。
SENSOR_TYPE_ID_AMBIENT_LIGHT	环境光传感器。
SENSOR_TYPE_ID_MAGNETIC_FIELD	磁场传感器。
SENSOR_TYPE_ID_CAPACITIVE	电容式传感器。
SENSOR_TYPE_ID_BAROMETER	气压计传感器。
SENSOR_TYPE_ID_TEMPERATURE	温度传感器。
SENSOR_TYPE_ID_HALL	霍尔传感器。
SENSOR_TYPE_ID_GESTURE	手势传感器。
SENSOR_TYPE_ID_PROXIMITY	接近光传感器。
SENSOR_TYPE_ID_HUMIDITY	湿度传感器。
SENSOR_TYPE_ID_PHYSICAL_MAX	物理传感器的最大类型 ID。
SENSOR_TYPE_ID_ORIENTATION	方向传感器。
SENSOR_TYPE_ID_GRAVITY	重力传感器。

成员名称	描述
SENSOR_TYPE_ID_LINEAR_ACCELERATION	线性加速度传感器。
SENSOR_TYPE_ID_ROTATION_VECTOR	旋转矢量传感器。
SENSOR_TYPE_ID_AMBIENT_TEMPERATURE	环境温度传感器。
SENSOR_TYPE_ID_MAGNETIC_FIELD_UNCALIBRATED	未校准磁场传感器。
SENSOR_TYPE_ID_GAME_ROTATION_VECTOR	游戏旋转矢量传感器。
SENSOR_TYPE_ID_GYROSCOPE_UNCALIBRATED	未校准陀螺仪传感器。
SENSOR_TYPE_ID_SIGNIFICANT_MOTION	有效运动传感器。
SENSOR_TYPE_ID_PEDOMETER_DETECTION	计步检测传感器。
SENSOR_TYPE_ID_PEDOMETER	计步传感器。
SENSOR_TYPE_ID_GEOMAGNETIC_ROTATION_VECTOR	地磁旋转矢量传感器。
SENSOR_TYPE_ID_HEART_RATE	心率传感器。
SENSOR_TYPE_ID_DEVICE_ORIENTATION	设备方向传感器。
SENSOR_TYPE_ID_WEAR_DETECTION	佩戴检测传感器。
SENSOR_TYPE_ID_ACCELEROMETER_UNCALIBRATED	未校准加速度计传感器。
SENSOR_TYPE_ID_MAX	传感器类型 ID 最大值。

【注意事项】

无。

【相关数据类型及接口】

- [2.3.2 SubscribeSensor](#)
- [2.3.3 UnsubscribeSensor](#)
- [2.3.5 ActivateSensor](#)
- [2.3.6 DeactivateSensor](#)
- [2.3.4 SetBatch](#)
- [2.3.7 SetMode](#)

2.4.2 SensorInfo

【说明】

设备支持的传感器信息。

【定义】

```
typedef struct SensorInfo {
    char sensorName[SENSOR_NAME_MAX_LEN];    /**< Sensor name */
    char vendorName[SENSOR_NAME_MAX_LEN];    /**< Sensor vendor */
    char firmwareVersion[VERSION_MAX_LEN];    /**< Sensor firmware version */
    char hardwareVersion[VERSION_MAX_LEN];    /**< Sensor hardware version */
    int32_t sensorTypeId;    /**< Sensor type ID */
    int32_t sensorId;        /**< Sensor ID */
    float maxRange;          /**< Maximum measurement range of the sensor */
    float precision;         /**< Sensor accuracy */
    float power;             /**< Sensor power */
} SensorInfo;
```

【成员】

成员名称	描述
sensorName	传感器名称。
vendorName	传感器厂商。
firmwareVersion	传感器固件版本号。
hardwareVersion	传感器硬件版本号。
sensorTypeId	唯一标识一个传感器类型，详情请参见 “2.4.1 SensorTypeId” 。
sensorId	传感器标识，硬件厂商自定义。
maxRange	传感器最大量程，暂不支持。
precision	传感器精度，暂不支持。
power	传感器功率，暂不支持。

【注意事项】

无。

【相关数据类型及接口】

- 2.3.1 GetAllSensors

2.4.3 SensorEvent

【说明】

传感器上报数据的结构。

【定义】

```
typedef struct SensorEvent {
    int32_t sensorTypeId; /**< Sensor type ID */
    int32_t version;      /**< Sensor algorithm version */
    int64_t timestamp;    /**< Time when sensor data was reported */
    uint32_t option;      /**< Sensor data options, including the measurement range and
accuracy */
    int32_t mode;         /**< Sensor data reporting mode (described in {@link SensorMode}) */
    uint8_t *data;        /**< Sensor data */
    uint32_t dataLen;     /**< Sensor data length */
} SensorEvent;
```

【成员】

成员名称	描述
sensorTypeId	唯一标识一个传感器类型，详情请参见“2.4.1 SensorTypeId”。
version	传感器算法版本，硬件厂商自定义。
timestamp	传感器数据上报时间。
option	传感器数据的精度量程等选项，暂不支持。
mode	传感器数据上报模式。
data	传感器数据。
dataLen	传感器数据长度。

【注意事项】

无。

【相关数据类型及接口】

- [2.4.4 RecordSensorCallback](#)

2.4.4 RecordSensorCallback

【说明】

传感器管理服务的传感器上报数据回调函数的定义。

【定义】

```
typedef void (*RecordSensorCallback)(SensorEvent *event);
```

【成员】

成员名称	描述
event	传感器上报数据的结构，详情请参见“ 2.4.3 SensorEvent ”。

【注意事项】

无。

【相关数据类型及接口】

- [2.4.6 SensorUser](#)

2.4.5 UserData

【说明】

传感器订阅的预留字段。

【定义】

```
typedef struct UserData {  
    char userData[SENSOR_USER_DATA_SIZE]; /**< Reserved for the sensor data subscriber  
*/  
} UserData;
```

【成员】

成员名称	描述
userData	传感器订阅者预留字段，最大

成员名称	描述
	SENSOR_USER_DATA_SIZE 字节。

【注意事项】

无。

【相关数据类型及接口】

- 2.4.6 SensorUser

2.4.6 SensorUser

【说明】

传感器的订阅用户。

【定义】

```
typedef struct SensorUser {
    char name[SENSOR_NAME_MAX_LEN]; /**< Name of the sensor data subscriber */
    RecordSensorCallback callback; /**< Callback for reporting sensor data */
    UserData *userData; /**< Reserved field for the sensor data subscriber */
} SensorUser;
```

【成员】

成员名称	描述
name	传感器订阅者的名称，最大 SENSOR_NAME_MAX_LEN 字节。
callback	传感器监听者的传感器数据上报回调函数，详情请参见 “2.4.4 RecordSensorCallback”。
userData	传感器订阅者预留字段，详情请参见 “2.4.5 UserData”。

【注意事项】

无。

【相关数据类型及接口】

- [2.3.2 SubscribeSensor](#)
- [2.3.3 UnsubscribeSensor](#)
- [2.3.4 SetBatch](#)
- [2.3.5 ActivateSensor](#)
- [2.3.6 DeactivateSensor](#)
- [2.3.7 SetMode](#)

2.4.7 SensorMode

【说明】

传感器的打开模式。

【定义】

```
typedef enum SensorMode {  
    SENSOR_DEFAULT_MODE = 0,    /**< Default data reporting mode */  
    SENSOR_REALTIME_MODE = 1,   /**< Real-time data reporting mode to report a group of  
data each time */  
    SENSOR_ON_CHANGE = 2,       /**< Real-time data reporting mode to report data upon status  
changes */  
    SENSOR_ONE_SHOT = 3,         /**< Real-time data reporting mode to report data only once */  
    SENSOR_FIFO_MODE = 4,        /**< FIFO-based data reporting mode to report data based on  
the <b>BatchCnt</b> setting */  
    SENSOR_MAX_MODE,             /**< Maximum sensor data reporting mode */  
} SensorMode;
```

【成员】

成员名称	描述
SENSOR_DEFAULT_MODE	传感器默认工作模式状态。
SENSOR_REALTIME_MODE	传感器实时工作模式状态，一组数据上报一次，暂不支持。
SENSOR_ON_CHANGE	传感器实时工作模式状态，状态变更上报一次，暂不支持。
SENSOR_ONE_SHOT	传感器实时工作模式状态，只上报一次，暂不支持。
SENSOR_FIFO_MODE	传感器缓存工作模式状态，根据配置的 BatchCnt 参数，每 BatchCnt 组数据上报一次，暂不支持。
SENSOR_MAX_MODE	传感器最大模式类型标识

【注意事项】

无。

【相关数据类型及接口】

- [2.3.7 SetMode](#)

2.4.8 AccelData

【说明】

加速度传感器的数据结构。

【定义】

```
typedef struct AccelData {  
    int16_t axisX;  
    int16_t axisY;  
    int16_t axisZ;  
} AccelData;
```

【成员】

成员名称	描述
axisX	加速度 X 轴。
axisY	加速度 Y 轴。
axisZ	加速度 Z 轴。

【注意事项】

无。

【相关数据类型及接口】

- [2.4.4 RecordSensorCallback](#)

2.4.9 GyroData

【说明】

角速度传感器的数据结构。

【定义】

```
typedef struct GyroData {
    int16_t axisX;
    int16_t axisY;
    int16_t axisZ;
} GyroData;
```

【成员】

成员名称	描述
axisX	加速度 X 轴。
axisY	加速度 Y 轴。
axisZ	加速度 Z 轴。

【注意事项】

无。

【相关数据类型及接口】

- [2.4.4 RecordSensorCallback](#)

2.4.10 PressureData

【说明】

气压传感器的数据结构。

【定义】

```
typedef struct PressureData {
    uint32_t pressure;
    uint32_t temperature;
    uint32_t dataValid;
} PressureData;
```

【成员】

成员名称	描述
pressure	气压计气压值。
temperature	气压计温度值。
dataValid	气压计数据有效性。

【注意事项】

无。

【相关数据类型及接口】

- [2.4.4 RecordSensorCallback](#)

2.4.11 SENSOR_NAME_MAX_LEN

【说明】

Sensor 命名的最大长度。

【定义】

```
#define SENSOR_NAME_MAX_LEN 16
```

【成员】

无

【注意事项】

无。

【相关数据类型及接口】

- [2.4.2 SensorInfo](#)

2.4.12 SENSOR_USER_DATA_SIZE

【说明】

传感器用户数据的大小。

【定义】

```
#define SENSOR_USER_DATA_SIZE 104
```

【成员】

无

【注意事项】

无。

【相关数据类型及接口】

- [2.4.5 UserData](#)

2.4.13 VERSION_MAX_LEN

【说明】

传感器版本号最大长度。

【定义】

```
#define VERSION_MAX_LEN 16
```

【成员】

无

【注意事项】

无。

【相关数据类型及接口】

- [2.4.2 SensorInfo](#)

2.5 错误码

表2-1 传感器子系统 API 错误码

错误代码	宏定义	描述
0	SENSOR_OK	成功。
-1	SENSOR_ERROR_UNKNOWN	失败。
-2	SENSOR_ERROR_INVALID_ID	无效的 ID。
-3	SENSOR_ERROR_INVALID_PARAM	无效的参数。

2.6 Sensor 驱动对接说明

传感器子系统通过硬件抽象层屏蔽硬件差异，负责硬件驱动对接管理，BrandyK100 上默认使用模拟 Sensor 驱动实现 EmulatorVendorImpl，在已经完成适配 Sensor 硬件驱动的情况下，可以切换到 SensorChipVendorImpl 使用。

传感器驱动对接步骤：

步骤 1 按照 HDI 接口实现对应的 SensorChipVendorImpl，可参考 EmulatorVendorImpl。

接口路径：openharmony\drivers\peripheral\sensor\interfaces\include

步骤 2 编译配置组件切换。

修改路径：build\config\target_config\common_config.py

在'ohos_set'中将 sensor_hdi 修改为 sensor_hdi_chip。

-----结束

2.7 开发指导

2.7.1 获取设备支持的 Sensor 列表

```
typedef struct {
    bool isStarted;
    SensorInfo *sensorInfo;
    int32_t snsCnt;
} SensorSampleContext;

static SensorSampleContext g_sensorContext = {
    .isStarted = false,
    .sensorInfo = NULL,
    .snsCnt = 0,
};

if (g_sensorContext.snsCnt != 0 || g_sensorContext.sensorInfo != NULL) {
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "sensorInfo is Got\n");
    return SENSOR_OK;
}

int32_t errCode = GetAllSensors(&g_sensorContext.sensorInfo, &g_sensorContext.snsCnt);
if (errCode != SENSOR_OK) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorInfo failed:%d\n", errCode);
    return errCode;
}
```

2.7.2 创建传感器订阅回调函数及传感器数据解析

```
/******
```

加速度传感器回调函数及数据解析

```

/*****/
void RecordAccelSensorCallback(SensorEvent *event)
{
    if (event->data == NULL || event->dataLen == 0 || event->dataLen != sizeof(AccelData)) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SensorEvent data invalid\n");
        return;
    }
    if (event->sensorTypeId != SENSOR_TYPE_ID_ACCELEROMETER) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorTypeId %d is not
ACCELEROMETER\n", event->sensorTypeId);
        return;
    }
    AccelData *accelData = (AccelData *)event->data;
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP,
        "accelData axisX:%d axisY:%d axisZ:%d \n",
        accelData->axisX,
        accelData->axisY,
        accelData->axisZ);
}

```

气压计传感器回调函数及数据解析

```

/*****/
void RecordBarSensorCallback(SensorEvent *event)
{
    if (event->data == NULL || event->dataLen == 0 || event->dataLen != sizeof(PressureData)) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SensorEvent data invalid\n");
        return;
    }
    if (event->sensorTypeId != SENSOR_TYPE_ID_BAROMETER) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorTypeId %d is not
BAROMETER\n", event->sensorTypeId);
        return;
    }
    PressureData *pressureData = (PressureData *)event->data;
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP,
        "pressureData pressure:%u temperature:%u dataValid:%u \n",
        pressureData->pressure,
        pressureData->temperature,
        pressureData->dataValid);
}

```

陀螺仪传感器回调函数及数据解析


```

/*****/
void RecordCyroSensorCallback(SensorEvent *event)
{
    if (event->data == NULL || event->dataLen == 0 || event->dataLen != sizeof(GyroData)) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SensorEvent data invalid\n");
        return;
    }
    if (event->sensorTypeId != SENSOR_TYPE_ID_GYROSCOPE) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorTypeId %d is not
GYROSCOPE\n", event->sensorTypeId);
        return;
    }
    GyroData *gyroData = (GyroData *)event->data;
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP,
        "gyroData axisX:%d axisY:%d axisZ:%d\n",
        gyroData->axisX,
        gyroData->axisY,
        gyroData->axisZ);
}
/*****/

```

心率传感器回调函数及数据解析

```

/*****/
void RecordHrSensorCallback(SensorEvent *event)
{
    if (event->data == NULL || event->dataLen == 0 || event->dataLen != sizeof(uint32_t)) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SensorEvent data invalid\n");
        return;
    }
    if (event->sensorTypeId != SENSOR_TYPE_ID_HEART_RATE) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorTypeId %d is not
HEART_RATE\n", event->sensorTypeId);
        return;
    }
    uint32_t heartRate = *(uint32_t *)event->data;
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "heartRate:%u", heartRate);
}
/*****/

```

佩戴检测传感器回调函数及数据解析

```

/*****/
void RecordObsSensorCallback(SensorEvent *event)
{
    if (event->data == NULL || event->dataLen == 0 || event->dataLen != sizeof(uint8_t)) {

```

```

        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SensorEvent data invalid\n");
        return;
    }
    if (event->sensorTypeId != SENSOR_TYPE_ID_WEAR_DETECTION) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorTypeId %d is not
WEAR_DETECTION\n", event->sensorTypeId);
        return;
    }
    uint8_t isonbody = *event->data;
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "isonbody:%u", isonbody);
}
/*****

计步器传感器回调函数及数据解析

*****/
void RecordScSensorCallback(SensorEvent *event)
{
    if (event->data == NULL || event->dataLen == 0 || event->dataLen != sizeof(uint32_t)) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SensorEvent data invalid\n");
        return;
    }
    if (event->sensorTypeId != SENSOR_TYPE_ID_PEDOMETER) {
        WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "sensorTypeId %d is not
PEDOMETER\n", event->sensorTypeId);
        return;
    }
    uint32_t steps = *(uint32_t *)event->data;
    WEARABLE_LOGI(WEARABLE_LOG_MODULE_APP, "steps:%u", steps);
}

```

2.7.3 创建传感器用户

```

SensorUser sensorUser;

sensorUser.callback = SensorDataCallbackImpl; //成员变量callback指向创建的回调方法

```

2.7.4 使能传感器

```

SensorUser sensorUser;

sensorUser.callback = SensorDataCallbackImpl; //成员变量callback指向创建的回调方法

int32_t errCode = ActivateSensor(sensorTypeId, &sensorUser);
if (errCode != SENSOR_OK) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "ActivateSensor failed:%d\n",
errCode);
}

```

```
    return errCode;
}
```

2.7.5 订阅传感器数据

```
SensorUser sensorUser;
sensorUser.callback = SensorDataCallbackImpl; //成员变量callback指向创建的回调方法
int32_t errCode = SubscribeSensor(sensorTypeId, &sensorUser);
if (errCode != SENSOR_OK) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "SubscribeSensor failed:%d\n",
errCode);
    return errCode;
}
```

2.7.6 取消传感器数据订阅

```
SensorUser sensorUser;
sensorUser.callback = SensorDataCallbackImpl; //成员变量callback指向创建的回调方法
int32_t errCode = UnsubscribeSensor(sensorTypeId, &sensorUser);
if (errCode != SENSOR_OK) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "UnsubscribeSensor failed:%d\n",
errCode);
    return errCode;
}
```

2.7.7 去使能一个传感器

```
SensorUser sensorUser;
sensorUser.callback = SensorDataCallbackImpl; //成员变量callback指向创建的回调方法
int32_t errCode = DeactivateSensor(sensorTypeId, &sensorUser);
if (errCode != SENSOR_OK) {
    WEARABLE_LOGE(WEARABLE_LOG_MODULE_APP, "DeactivateSensor failed:%d\n",
errCode);
    return errCode;
}
```

3 HTTP 组件

3.1 Native API 参考

3.2 数据类型，数据结构

3.3 错误码

3.1 Native API 参考

提供一下 API：

- HttpClientGetRequest：提供基于 URL 的 GET 请求，同步阻塞接口。
- HttpClientPostRequest：提供基于 URL 的 POST 请求，同步阻塞接口。
- HttpClientHeadRequest：提供基于 URL 的 HEAD 请求，同步阻塞接口。
- HttpClientPutRequest：提供基于 URL 的 PUT 请求，同步阻塞接口。
- HttpClientDelete：提供基于 URL 的 DELETE 请求，同步阻塞接口。
- HttpClientConn：基于 URL 建立 HTTP/HTTPS 连接。
- HttpClientSend：发送 HTTP/HTTPS 请求接口。
- HttpClientRecvResponse：从远端接收 response。
- HttpClientClose：关闭 HTTP 连接。
- HttpClientGetResponseCode：获取 HTTP response code。
- HttpClientSetCustomHeader：配置自定义的 HTTP header。

3.1.1 HttpClientGetRequest

【描述】

提供基于 URL 的 GET 请求，同步阻塞接口。

【语法】

```
HTTTPC_RESULT HttpClientGetRequest(HttpClient *client, const char *url, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入
clientData	clientData 是指针指向 HttpClientData 结构体，保存返回的数据。	输入，输出

【返回值】

返回值	描述
HTTTPC_RESULT	0：成功。 - 非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

3.1.2 HttpClientPostRequest

【描述】

提供基于 URL 的 POST 请求，同步阻塞接口。

【语法】

```
HTTTPC_RESULT HttpClientPostRequest(HttpClient *client, const char *url, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入
clientData	clientData 是指针指向 HttpClientData 结构体，保存返回的数据。	输入，输出

【返回值】

返回值	描述
HTTTPC_RESULT	0: 成功。 - 非 0: 失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

3.1.3 HttpClientHeadRequest

【描述】

提供基于 URL 的 HEAD 请求，同步阻塞接口。

【语法】

```
HTTTPC_RESULT HttpClientHeadRequest(HttpClient *client, const char *url, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入
clientData	clientData 是指针指向	输入，输出

参数名称	描述	输入/输出
	HttpClientData 结构体，保存返回的数据。	

【返回值】

返回值	描述
HTTPC_RESULT	0：成功。 - 非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

3.1.4 HttpClientPutRequest

【描述】

提供基于 URL 的 PUT 请求，同步阻塞接口。

【语法】

```
HTTPC_RESULT HttpClientPutRequest(HttpClient *client, const char *url, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入
clientData	clientData 是指针指向 HttpClientData 结构体，保存返回的数据。	输入，输出

【返回值】

返回值	描述
HTTPC_RESULT	0：成功。 - 非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

3.1.5 HttpClientDelete

【描述】

提供基于 URL 的 Delete 请求，同步阻塞接口。

【语法】

```
HTTPC_RESULT HttpClientDeleteRequest(HttpClient *client, const char *url, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入
clientData	clientData 是指针指向 HttpClientData 结构体，保存返回的数据。	输入，输出

【返回值】

返回值	描述
HTTPC_RESULT	0：成功。

返回值	描述
	非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

无。

3.1.6 HttpClientConn

【描述】

基于 URL 建立 HTTP/HTTPS 连接。

【语法】

```
HTTTPC_RESULT HttpClientConn(HttpClient *client, const char *url);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入

【返回值】

返回值	描述
HTTTPC_RESULT	0：成功。 - 非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

该接口和 HttpClientConn、HttpClientSend、HttpClientRecvResponse、HttpClientClose 组合使用。

3.1.7 HttpClientSend

【描述】

发送 HTTP/HTTPS 请求接口。

【语法】

```
HTTTPC_RESULT HttpClientSend(HttpClient *client, const char *url, int method, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
url	请求的 URL。	输入
method	HTTP 请求方法。	输入
clientData	clientData 是指针指向 HttpClientData 结构体，发送的数据。	输入

【返回值】

返回值	描述
HTTTPC_RESULT	0：成功。 - 非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

该接口和 HttpClientConn、HttpClientSend、HttpClientRecvResponse、HttpClientClose 组合使用。

3.1.8 HttpClientRecvResponse

【描述】

从远端接收 response。

【语法】

```
HTTPC_RESULT HttpClientRecvResponse(HttpClient *client, HttpClientData *clientData);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
clientData	clientData 是指针指向 HttpClientData 结构体，发送的数据。	输出

【返回值】

返回值	描述
HTTPC_RESULT	0：成功。 - 非 0：失败，请参考“3.3 错误码”章节说明。

【芯片差异】

无。

【注意】

该接口和 HttpClientConn、HttpClientSend、HttpClientRecvResponse、HttpClientClose 组合使用。

3.1.9 HttpClientClose

【描述】

关闭 HTTP 连接。

【语法】

```
void HttpClientClose(HttpClient *client);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入

【返回值】

返回值	描述
void	返回空

【芯片差异】

无。

【注意】

该接口和 HttpClientConn、HttpClientSend、HttpClientRecvResponse、HttpClientClose 组合使用。

3.1.10 HttpClientGetResponseCode

【描述】

获取 HTTP response code。

【语法】

```
int HttpClientGetResponseCode(HttpClient *client);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入

【返回值】

返回值	描述
int	HTTP 服务端返回的状态码。

【芯片差异】

无。

【注意】

无。

3.1.11 HttpClientSetCustomHeader

【描述】

配置自定义的 HTTP header。

【语法】

```
void HttpClientSetCustomHeader(HttpClient *client, char *header);
```

【参数】

参数名称	描述	输入/输出
client	client 指针指向 HttpClient 结构体。	输入
header	header 字符串指针。	输入

【返回值】

返回值	描述
void	返回空

【芯片差异】

无。

【注意】

无。

3.2 数据类型，数据结构

3.2.1 HTTP Request 类型

【说明】

HTTP Request 类型。

【定义】

```
/** @brief http request type */
typedef enum {
    HTTP_DELETE,
    HTTP_GET,
    HTTP_HEAD,
    HTTP_POST,
    HTTP_PUT
} HTTP_REQUEST_TYPE;
```

【成员】

成员名称	描述
HTTP_DELETE	HTTP Delete 方法。
HTTP_GET	HTTP Get 方法。
HTTP_HEAD	HTTP HEAD 方法。
HTTP_POST	HTTP POST 方法。
HTTP_PUT	HTTP PUT 方法。

【注意事项】

无。

3.2.2 HTTP Client 结构体

【说明】

HTTP Client 结构体。

【定义】

```

/** @brief This structure defines the HttpClient structure */
typedef struct {
    int socket;           /**< socket ID */
    int remotePort;       /**< hTTP or HTTPS port */
    int responseCode;     /**< response code */
    char *header;         /**< request custom header */
    char *authUser;       /**< username for basic authentication */
    char *authPassword;   /**< password for basic authentication */
    bool isHttp;          /**< http connection? if 1, http; if 0, https */
    #if CONFIG_HTTP_SECURE
    const char *serverCert; /**< server certification */
    const char *clientCert; /**< client certification */
    const char *clientPk;  /**< client private key */
    int serverCertLen;     /**< server certification lenght, server_cert buffer size */
    int clientCertLen;     /**< client certification lenght, client_cert buffer size */
    int clientPkLen;       /**< client private key lenght, client_pk buffer size */
    void *ssl;             /**< ssl content */
    #endif
} HttpClient;

```

【成员】

成员名称	描述
socket	传输套接字 ID，非入参。
remotePort	远端端口号，非入参。
responseCode	服务端返回 HTTP 状态码，出参。
header	发送请求 header 指针，可选入参。
authUser	用于认证的用户名，可选入参。
authPassword	用于认证的密码，可选入参。
isHttp	是否是 http 连接，0: https; 1: http，非入参。
serverCert	服务端证书内容指针，可选入参。
clientCert	客户端证书内容指针，可选入参。
clientPk	客户端私钥。可选入参。
serverCertLen	服务端证书大小，可选入参。

成员名称	描述
clientCertLen	客户端证书大小，可选入参。
ssl	ssl 内部指针，非入参。

【注意事项】

无

3.2.3 HttpClientData 结构体

【说明】

HTTP ClientData 结构体。

【定义】

```
/** @brief This structure defines the HTTP data structure. */
typedef struct {
    bool isMore;           /**< indicates if more data needs to be retrieved. */
    bool isChunked;        /**< response data is encoded in portions/chunks.*/
    int method;            /**< http method. */
    int retrieveLen;        /**< content length to be retrieved. */
    int responseContentLen; /**< response content total length. */
    int contentBlockLen;    /**< the content length of one block. */
    int postBufLen;         /**< post data length. */
    int responseBufLen;     /**< response body buffer length. */
    int headerBufLen;       /**< response head buffer length. */
    char *postContentType;  /**< content type of the post data. */
    char *postBuf;          /**< user data to be posted. */
    char *responseBuf;      /**< buffer to store the response body data. */
    char *headerBuf;        /**< buffer to store the response head data. */
    bool isRedirected;      /**< redirected URL? if 1, has redirect url; if 0, no redirect url */
    char *redirectUrl;      /**< redirect url when got http 3** response code. */
} HttpClientData;
```

【成员】

成员名称	描述
isMore	出参，表示当前是否有更多数据需要获取。
isChunked	非入出参，内部使用。

成员名称	描述
method	非入参，HTTP 请求方法。
retrieveLen	非出参，内部使用。
responseContentLen	服务端返回 response body 数据总大小，出参。
contentBlockLen	本次请求返回的 body 数据大小，出参。
postBufLen	POST 请求，POST 缓冲区大小，入参。
responseBufLen	用于获取服务端 response body 的缓存区大小，入参，要求大于等于 2048Byte。
headerBufLen	用户获取服务端返回的 header 缓冲区大小，入参，要求大于等于 2048Byte。
postContentType	POST 数据内容类型，可选入参。
postBuf	POST 请求，POST 缓冲区大小，入参。
responseBuf	用于获取服务端 response body 的缓存区指针，入参。
headerBuf	ssl 内部指针，非入参。
isRedirected	是否重定位 URL，出参。
redirectUrl	重定位的 URL，出参。

【注意事项】

无

3.3 错误码

```
/** @brief http error code */
typedef enum {
    HTTP_EAGAIN    = 1,  /**< more data to retrieved */
    HTTP_SUCCESS   = 0,  /**< operation success */
    HTTP_ENOBUFS   = -1, /**< buffer error */
    HTTP_EARG      = -2, /**< illegal argument */
    HTTP_ENOTSUPP  = -3, /**< not support */
    HTTP_EDNS      = -4, /**< DNS fail */
}
```

```

HTTP_ECONN    = -5, /**< connect fail          */
HTTP_ESEND    = -6, /**< send packet fail       */
HTTP_ECLSD    = -7, /**< connect closed        */
HTTP_ERECEV   = -8, /**< recv packet fail      */
HTTP_EPARSE   = -9, /**< url parse error       */
HTTP_EPROTO   = -10, /**< protocol error       */
HTTP_EUNKOWN   = -11, /**< unknown error       */
HTTP_ETIMEOUT = -12, /**< timeout              */
} HTTPC_RESULT;

```

表3-1 HTTP API 错误码

错误码	错误解释
HTTP_EAGAIN = 1	更多数据等待获取。
HTTP_SUCCESS = 0	成功。
HTTP_ENOBUFS = -1	无内存错误。
HTTP_EARG = -2	非法参数。
HTTP_ENOTSUPP = -3	功能不支持。
HTTP_EDNS = -4	DNS 失败。
HTTP_ECONN = -5	连接失败。
HTTP_ESEND = -6	发送报文失败。
HTTP_ECLSD = -7	连接关闭。
HTTP_ERECEV = -8	接收报文失败。
HTTP_EPARSE = -9	URL 解析失败。
HTTP_EPROTO = -10	协议错误。
HTTP_EUNKOWN = -11	未知错误。
HTTP_ETIMEOUT = -12	超时错误。